

AGORA Evidence Chain

**Ein Code-Audit zu Provenance und Anti-Self-Confirmation in einer
Multi-Agent-Simulationsplattform**

Alexander Schneider

ORCID 0009-0008-7552-1819

2026-08-09

Repository `arn01d87/agora` – `github.com/arnold87/agora`, Branch
`feat/1152-provenance-retrieval`, HEAD `7e42ae34`

Version 0.9.3 Stability Beta

Begleitendes Software-Artefakt: Zenodo `21855023`

Diese Arbeit untersucht am Code-Audit eines realen Multi-Agent-Systems (AGORA, Version 0.9.3 Stability Beta), welche Bestandteile der Evidence-Pipeline einen *tatsächlich überprüfba-*
ren Erkenntnisgewinn gegenüber einem einzelnen starken Large-Language-Model-Prompt erzeugen, und welche lediglich zusätzliche Komplexität, Kosten oder scheinbare Evidenz. Methodisch verknüpft der Beitrag eine statische Code-Audit-Pipeline (acht parallele Subagenten mit Code-Review-Graph-Unterstützung) mit einer normativen Bewertung anhand eines zehn-dimensionalen Scorerasters. Im Mittelpunkt steht die Rekonstruktion der Evidence-Chain vom Seed-Dokument bis zum Claim über 17 Stufen, einschließlich sieben code-verifizierter Provenance-Verlustpunkten. Die Ergebnisse zeigen, dass AGORA als *ehrlich konstruiertes, aber empirisch unbewiesenes* System zu lesen ist: Der Engineering-Score liegt bei 7,0/10, der Evidence-Research-Score bei 3,5/10; ein Referenzlauf lieferte null validierte Claims. Die Diskussion arbeitet fünf Kernkontroversen heraus und priorisiert Po-Maßnahmen (Seed-Provenance, Interview-Binding, Konzepttrennung der Confidence-Scores), die den Mehrwert der Pipeline überführbar machen sollen. Der Beitrag ist sowohl Diagnose als auch Bauplan.

[english] This paper investigates, via a code audit of a real multi-agent system (AGORA, v0.9.3 Stability Beta), which parts of the evidence pipeline yield a *verifiable* epistemic gain over a single strong large language model prompt, and which add complexity, cost, or apparent evidence only. Methodologically, the work links a static code-audit pipeline (eight parallel sub-agents, supported by a code-review-graph) with a normative evaluation along a ten-dimensional scoring scheme. The core artefact is a reconstruction of the evidence chain from seed document to claim across seventeen stages, including seven code-verified provenance loss points. The findings read AGORA as an *honestly engineered but empirically unproven* system: engineering score 7.0/10, evidence-research score 3.5/10, one reference run producing zero validated claims. The discussion isolates five key controversies and prioritises Po measures (seed-document provenance, interview-binding fix, separation of confidence concepts) that should make the pipeline's added value observable. The contribution is simultaneously diagnosis and blueprint.

Inhaltsverzeichnis

I. Kontext und Forschungsfrage	9
1. Einleitung und Forschungsfrage	10
1.1. Motivation	10
1.2. Forschungsfrage	10
1.3. Beitrag	11
1.4. Gliederung	11
1.5. Begriffliche Abgrenzung	11
1.6. Hinweis zum Reproduktionsstand	12
II. Stand der Forschung	13
2. Stand der Forschung	14
2.1. Synthetische Personas und generative Agenten	14
2.2. Multi-Agent-Debate und Self-Consistency	14
2.3. GraphRAG und Retrieval-Grounding	15
2.4. Halluzinationserkennung und Kalibrierung	15
2.5. Synthetische Daten und Model-Collapse	15
2.6. Analytische Flexibilität und Reproduzierbarkeit	15
2.7. Synthese: Wo setzt AGORA an?	16
III. System und Pipeline	17
3. AGORA: System und Pipeline	18
3.1. Architekturüberblick	18
3.2. Die Evidence-Chain	18
3.3. Die EvidenceSourceKind-Taxonomie	18
3.4. Zweistufiges Claim-Binding	19
3.5. Anti-Self-Confirmation-Mechanismen	19
3.6. Simulations-Subprozess	20
3.7. Vom Code zum Paper: Annotationsstrategie	20

IV. Methodik	22
4. Methodik	23
4.1. Übersicht: Audit-Pipeline	23
4.2. Code-Review-Graph als Strukturspeicher	23
4.3. Verifikation am Code	24
4.4. Counter-Review	24
4.5. Reproduzierbarkeit der Methode	25
4.6. Einschränkungen	25
4.7. Annotation der Belege	25
V. Ergebnisse	26
5. Ergebnisse	27
5.1. Referenzlauf und Baseline	27
5.2. 10-Dimensionen-Bewertung	27
5.3. Failure-Mode-Analyse	27
5.4. Provenance-Verlustpunkte	27
5.5. Prompt-vs-AGORA-Matrix	28
5.6. Novelty-Matrix	28
5.7. Anti-Self-Confirmation-Bewertung	29
5.8. Referenzläufe im Vergleich	30
5.9. Zwischenfazit	30
VI. Diskussion	33
6. Diskussion	34
6.1. Fünf Kernkontroversen	34
6.1.1. Kontroverse 1: Evidence-Pipeline als Differenzierer?	34
6.1.2. Kontroverse 2: Anti-Self-Confirmation über Prompt-Level?	34
6.1.3. Kontroverse 3: o validierte Claims = kein Mehrwert?	34
6.1.4. Kontroverse 4: OASIS-Mehrwert?	35
6.1.5. Kontroverse 5: ADR-Honesty = Ersatz für Mehrwert?	35
6.2. Engineering-Score vs. Evidence-Research-Score	35
6.3. Warum greift das Gate aktuell ins Leere?	35
6.4. Limitationen	36
6.5. Vergleich mit dem Stand der Forschung	36
6.6. Konsequenzen für andere Multi-Agent-Systeme	36
6.7. Methodische Lehre	37

VII. Schluss	38
7. Schluss	39
7.1. Beantwortung der Forschungsfrage	39
7.2. Ausblick: Priorisierte Roadmap	39
7.3. Verfügbarkeit und Reproduzierbarkeit	40
7.4. Schlussbemerkung	40
A. Anhang A: Verifikationsprotokoll	41
A.1. Verifikations-Spot-Checks	41
A.2. Failure-Mode-Tabelle (vollständig)	41
A.3. Citation-Registry	41
A.4. Der zurückgezogene Befund C56	43
Literaturverzeichnis	44

Abbildungsverzeichnis

3.1.	Rekonstruierte Evidence-Chain vom Seed-Dokument bis zum Claim. Die sieben mit  markierten Stufen sind code-verifizierte Provenance-Verlustpunkte. . . .	21
4.1.	Subagent-Topologie der Audit-Pipeline. Der Lead bleibt im Kontrollthread; jeder Subagent arbeitet isoliert und liefert eine Note zurück.	23

Tabellenverzeichnis

3.1.	Anti-Self-Confirmation-Mechanismen in AGORA; Wirkung und Code-Anker. .	20
5.1.	10-Dimensionen-Bewertung von AGORA zum Auditzeitpunkt.	28
5.2.	Auswahl kritischer Failure-Modes (vollständige Liste im Anhang).	29
5.3.	Sieben code-verifizierte Provenance-Verlustpunkte in der Evidence-Chain. . .	30
5.4.	Prompt-vs-AGORA-Matrix: Welche Mechanismen sind strukturell (X) und damit nicht durch einen stärkeren System-Prompt ersetzbar?	31
5.5.	Novelty-Matrix: Einordnung zentraler AGORA-Eigenschaften auf den Stufen Lo-L4.	31
5.6.	Bewertung der Anti-Self-Confirmation-Mechanismen gegenüber den in der Literatur dokumentierten Risiken.	32
5.7.	Vergleich der Referenzläufe. R1 dokumentiert den Vor-Fix-Zustand; R2 den Pre-Fix-Stand des Interview-Binding-Defekts; R4 ist der in ADR-0011 als Beleg zitierte Report.	32
A.1.	Verifikations-Spot-Checks (Auszug). Die vollständige Liste enthält 33 Belege; jede Kernaussage ruht auf mindestens zwei Belegen.	41
A.2.	Vollständige Failure-Mode-Tabelle (16 Einträge).	42

Teil I.

Kontext und Forschungsfrage

1. Einleitung und Forschungsfrage

1.1. Motivation

Multi-Agent-Frameworks auf Basis großer Sprachmodelle werden zunehmend als Werkzeug für die Marktanalyse, Stakeholder-Simulation und Szenarienmodellierung eingesetzt [1–3]. Die Versprechen sind hoch: synthetische Populationen sollen reale Zielgruppen approximieren, Multi-Agent-Debate soll die Factuality [4] erhöhen, und Selbstkonsistenz [5] soll gegen Halluzination absichern. Die Literatur zeigt jedoch zugleich, dass Konsistenz nicht mit Korrektheit gleichgesetzt werden darf [6], dass rekursives Training auf synthetischen Daten zu Model-Collapse führt [7], dass Inline-Zitationen häufig „cited but not verified“ sind [8] und dass die Validität von LLM-Personas gegen reale Populationen eine offene empirische Frage bleibt [9, 10].

Diese Arbeit setzt genau dort an: Sie untersucht am konkreten Code eines realen Multi-Agent-Systems (AGORA), welche Bausteine der Evidence-Pipeline tatsächlich einen *prüfbaren* Mehrwert liefern – und welche lediglich zusätzliche Komplexität erzeugen, ohne den Anspruch einzulösen.

1.2. Forschungsfrage

Die zentrale Forschungsfrage lautet:

Welche Teile von AGORA erzeugen einen tatsächlich überprüfbaren Erkenntnisgewinn gegenüber einem einzelnen starken LLM-Prompt – und welche Teile erzeugen lediglich zusätzliche Komplexität, Kosten oder scheinbare Evidenz?

Drei Unterfragen strukturieren die Antwort:

1. **Provenance:** Lässt sich der Pfad vom Seed-Dokument bis zum ausgelieferten Claim vollständig rekonstruieren – und an welchen Stellen geht die Herkunft verloren?
2. **Anti-Self-Confirmation:** Welche Mechanismen verhindern, dass synthetische Populationen sich gegenseitig bestätigen, ohne dass diese Mechanismen kalibriert oder extern validiert wären?
3. **Empirischer Mehrwert:** Lässt sich an einem realen Referenzlauf zeigen, dass das System über einen Single-Prompt hinaus zusätzliche, *geprüfte* Aussagen liefert – oder erstickt es an seiner eigenen Strenge?

1.3. Beitrag

Der Beitrag dieser Arbeit ist dreigliedrig:

- **Methodisch** wird eine reproduzierbare Code-Audit-Pipeline vorgestellt, die parallele Subagenten mit Code-Review-Graph-Unterstützung und einer normativen Bewertung verknüpft. Die Pipeline ist nicht auf AGORA beschränkt; sie ist auf jedes Multi-Agent-System übertragbar, dessen Code zugänglich ist.
- **Diagnostisch** liefert die Arbeit eine Rekonstruktion der Evidence-Chain vom Seed-Dokument bis zum Claim, mit sieben code-verifizierten Provenance-Verlustpunkten, einer 10-Dimensionen-Bewertung und einer priorisierten Roadmap.
- **Normativ** wird ein Antwortrahmen vorgeschlagen, mit dem sich die Frage „Liefert ein LLM-System einen messbaren Mehrwert?“ überhaupt erst formulieren lässt – inklusive der Trennung in *engineering score* (Konstruktion) und *evidence-research score* (Empirie).

1.4. Gliederung

Die Arbeit gliedert sich in sieben Kapitel. Kapitel 2 arbeitet den Stand der Forschung anhand von fünf Themenclustern auf und benennt die Forschungslücke, in die der Beitrag fällt. Kapitel 3 rekonstruiert die Architektur und die Evidence-Chain von AGORA. Kapitel 4 beschreibt die Audit-Pipeline. Kapitel 5 trägt die Befunde aus Code-Audit, Referenzlauf und Bewertung zusammen. Kapitel 6 diskutiert fünf Kernkontroversen. Kapitel 7 beantwortet die Forschungsfrage und priorisiert den Ausblick. Anhang A dokumentiert die Verifikations-Spot-Checks und die Methodik der Citation-Registry.

1.5. Begriffliche Abgrenzung

Drei Begriffe werden im Folgenden streng in der hier definierten Bedeutung verwendet:

Seed-Dokument Ein vom Nutzer hochgeladenes Korpus-Dokument (PDF, Markdown oder Text), das als einzige *echte externe Quelle* in die Pipeline eingeht. Alle anderen `EvidenceSourceKind`-Werte [11] sind entweder LLM-generiert, synthetisch oder abgeleitet.

Validierter Claim Ein Claim, der eine `supports_claim = True`-Zuordnung von der zweistufigen Entailment-Stufe erhalten hat. Diese Stufe kombiniert eine Cosine-Retrieval-Bewertung mit regelbasierten deterministischen Prüfungen [12].

Provenance-Verlust Ein Punkt in der Pipeline, an dem die Identität einer Information – ihre doc-ID, chunk-ID oder Herkunftsquelle – entweder wegfällt oder nicht in einen späteren Verarbeitungsschritt übernommen wird.

1.6. Hinweis zum Reproduktionsstand

Der Audit wurde am Repo-Stand `7e42ae34` durchgeführt. Zwischen Stand des Audits und dieser Niederschrift liegen zwei Merges in den Hauptzweig, die zentrale Befunde teils entschärfen: ADR-0013 Slice 1 Teil A (PR #1155) und der Interview-Binding-Fix (Commit `d7d9f0a4`, PR #1151). Die Stellen sind in der Diskussion ausdrücklich markiert.

Teil II.

Stand der Forschung

2. Stand der Forschung

Der Stand der Forschung wird in sechs Themencluster gegliedert, die für die Bewertung von AGORA direkt relevant sind. Pro Cluster werden zwei bis vier Schlüsselarbeiten referenziert und auf die Forschungslücke zugespitzt, die diese Arbeit schließen hilft.

2.1. Synthetische Personas und generative Agenten

Die zentralen Vorarbeiten stammen von [1], die in der Smallville-Simulation 25 generative Agenten über zwei Tage agieren lassen und qualitative Plausibilität, nicht aber populationsstatistische Repräsentativität nachweisen. [2] zeigen, dass LLM-Persona-Conditioning Antwortverteilungen replizieren kann, betonen aber die Prompt-Abhängigkeit. [3] demonstriert qualitative Replikation ökonomischer Verhaltensweisen. [9] untersuchen systematisch die Replikation realer Survey-Daten und identifizieren persistenten Bias.

Forschungslücke: Es existiert kein kanonisches Protokoll, um LLM-Personas systematisch gegen reale Populationsstatistiken zu validieren [13]. Übertragbarkeit auf DACH-Populationen ist eine offene empirische Frage.

2.2. Multi-Agent-Debate und Self-Consistency

[4] zeigen, dass Multi-Agent-Debate die Factuality gegenüber einzelnen Modellen verbessert, dass der Effekt jedoch bei großen Modellen sättigt. [5] liefern das methodische Fundament für Self-Consistency-Ansätze; [6] relativieren deren Wert durch den Befund, dass Konsistenz nur bedingt mit Korrektheit korreliert. [14] zeigen, dass Kollektivreasoning am Hidden-Profile-Problem systematisch versagt (30,1 % vs. 80,7 % für einen Einzelagent mit vollständiger Information). [15] klassifizieren Fehlermodi in Multi-Agent-Systemen und stellen fest, dass Benchmark-Gewinne oft minimal sind.

Forschungslücke: Studien, die Multi-Agent-Debate *mit aktiver Quellenverifikation* kombinieren, sind selten; [16] ist eine Ausnahme.

2.3. GraphRAG und Retrieval-Grounding

[17] zeigen, dass GraphRAG bei globalen, query-fokussierten Zusammenfassungen Standard-RAG übertrifft. Inline-Zitationen sind, wie [8] nachweisen, häufig fehlerhaft oder irrelevant – ein direkter Bezugspunkt zu Evidence-Gating [18].

Forschungslücke: Vergleichende Evaluationen zwischen GraphRAG und Standard-RAG bei festem Budget sind rar; Effizienz-Varianten wie E²GraphRAG oder Practical GraphRAG sind erst seit 2025/2026 verfügbar.

2.4. Halluzinationserkennung und Kalibrierung

[19] formuliert die Warnung vor Homogenität, Halluzination und fehlender Validierung in der sozialwissenschaftlichen Anwendung generativer KI. [6] zeigt, dass Selbstvertrauen und Korrektheit entkoppelt sind.

Forschungslücke: Verbindung von Halluzinationserkennung mit prozessualen Gating-Mechanismen – wie AGORA sie implementiert [18] – ist methodisch unterentwickelt.

2.5. Synthetische Daten und Model-Collapse

[7] zeigen, dass rekursives Training auf synthetisch generierten Daten die Verteilung verengt und schließlich zum Vergessen führt. Diese Erkenntnis ist für AGORA zentral, da die Persona-Population vollständig synthetisch ist und über mehrere Simulations-Runden weiter aggregiert wird.

Forschungslücke: Langfristige Stabilität synthetischer Multi-Agent-Simulationen über Wochen oder Monate wird kaum systematisch gemessen.

2.6. Analytische Flexibilität und Reproduzierbarkeit

[10] untersuchen 252 Konfigurationen eines LLM-basierten Sozialmodells und zeigen, dass die Ergebnisse stark von Prompt, Sampling und Modellwahl abhängen – ein P-Hacking-Äquivalent.

Forschungslücke: Standardisierte Reproduzierbarkeitsprotokolle für LLM-basierte Simulationen fehlen.

2.7. Synthese: Wo setzt AGORA an?

AGORA sitzt methodisch am Schnittpunkt von drei Clustern: (1) *Provenance-Grounding* durch serverseitige Anchor-Erzwingung, (2) *Anti-Self-Confirmation* durch Echo-Chamber-Index und Confidence-Capping, und (3) *Halluzinationsprävention* durch zweistufiges Binding. Damit berührt es Forschungslücken, die in der Literatur benannt sind, ohne sie bislang geschlossen zu haben. Der vorliegende Beitrag macht diese Lücken am konkreten Code sichtbar und formuliert eine Roadmap, die sie schließen könnte.

Teil III.

System und Pipeline

3. AGORA: System und Pipeline

3.1. Architekturüberblick

AGORA ist eine lokal oder kontrolliert hybrid betreibbare Multi-Agent-Plattform für simulierte DACH-Zielgruppen-, Stakeholder- und Marktreaktionen. Die Plattform ist als Flask/Python-3.14-Backend mit Neo4j-Graph, Redis-Cache, OASIS- bzw. CAMEL-Simulations-Subprozess und Vue-3-Frontend strukturiert. Die Code-Basis umfasst zum Auditzeitpunkt etwa 3500 LOC im Backend-Service-Layer, 5500 LOC im OASIS-Subprozess und etwa 3900 LOC Graph-bezogenen Code. Der Reifegrad ist *0.9.3 Stability Beta*; das Ziel ist nach ROADMAP .md eine stabile Single-User-Version 1 . 0 . 0¹

Für die vorliegende Arbeit sind zwei Schichten zentral: die **Evidence-Pipeline** (vom Seed-Dokument zum Claim) und die **Simulations-Pipeline** (Persona-Population, OASIS-Aktionen, Echo-Index). Beide sind über die EvidenceSourceKind-Taxonomie [12] miteinander verbunden.

3.2. Die Evidence-Chain

Abbildung 3.1 rekonstruiert die Evidence-Chain vom Seed-Dokument bis zum ausgelieferten Claim in 17 Stufen. Die Rekonstruktion beruht auf statischer Code-Analyse von `backend/app/services/` und `backend/app/contracts/`, validiert durch die Code-Review-Graph-Werkzeuge. Sieben Stufen (☒) markieren code-verifizierte Provenance-Verlustpunkte.

3.3. Die EvidenceSourceKind-Taxonomie

Der Vertrag `backend/app/contracts/report_contract.py` definiert sechs Werte für den Enum `EvidenceSourceKind`:

- `seed_corpus` – echte externe Quelle aus dem hochgeladenen Korpus. Aktuell nur als Zielzustand in [11] spezifiziert; im Referenzlauf [20] nicht erreicht.
- `agent_quote` – Persona-Interview-Zitat. Provenance über `persona_id`, `stakeholder_group`, `quote`.

¹Status-Informationen aus AGENTS .md und ROADMAP .md im Repository.

- `agent_action` – sozialer Akt aus der Simulation (Post, Comment, Reshare). Provenance über `sim_action_log` (Plattform, Runde, Agent, Aktion, Zeitstempel).
- `graph_relation` – Fakt aus dem Neo4j-Graph. Aktuell nur syntaktisch referenzierbar; *keine* Dokumentidentität.
- `web_source` – URL aus externer Recherche. URL im Anker; Fetch ist echtzeitabhängig und kann verfallen.
- `inferred` – Standard-Fallback, wenn die Herkunft unbekannt ist. `Confidence`-Gewicht ist 0,0, d. h. ein Claim auf `inferred` endet als Hypothese.

Die Trennung ist die Grundlage für die nachfolgend beschriebene Pipeline – und zugleich der Punkt, an dem die sechs Werte in der praktischen Wirksamkeit sehr ungleich verteilt sind.

3.4. Zweistufiges Claim-Binding

Die `evidence_binder` und `evidence_entailment` (in `backend/app/services/`) implementieren ein zweistufiges Binding:

1. **Retrieval-Stage:** Cosine-Ähnlichkeit zwischen Claim-Vektor und Evidence-Vektor. Schwellen 0,55 (niedrige Confidence) und 0,65 (hohe Confidence).
2. **Entailment-Stage:** Regelbasierte deterministische Prüfung – SUPPORTED, CONTRADICTED, RELATED_ONLY oder INSUFFICIENT. Diese Stufe prüft Zahl, Bezugsgruppe, Menge und Modalität explizit vor dem Embedding und verhindert damit das in [12] belegte Defektmuster „61 % Zeitersparnis → 61 % bewerten positiv“.

Das `supports_claim`-Feld wird ausschließlich in der Entailment-Stufe gesetzt; dies ist im Code-Kommentar zu `evidence_entailment.py` ausdrücklich festgehalten. Diese Architektur ist zentral für das Verständnis des gesamten Audit-Befundes, weshalb sie in Kapitel 5 erneut aufgegriffen wird.

3.5. Anti-Self-Confirmation-Mechanismen

AGORA implementiert vier strukturelle Anti-Self-Confirmation-Mechanismen, die in der folgenden Tabelle zusammengefasst sind. Ausführlich werden sie in Kapitel 5 bewertet.

Mechanismus	Wirkung
Echo-Chamber-Index (<code>network_analytics.py</code>)	Misst intra-cluster-Anteil synthetischer Aktionen; drosselt Confidence bei Werten $> 0,75$ via <code>apply_echo_cap</code> .
DACH-Quoten-Kalibrierung	Verhindert willkürliche Persona-Population; IT-Anteil hard-gecappt $\leq 12\%$; DACH-Branchenanteile gegen Destatis WZ 2008 / Mikrozensus 2024 kalibriert.
Wording-Glossar (<code>_red_team_system_prompt</code>)	Verbietet Vorhersagesprache; Volltext-Snapshot; Red-Team-Enforcement post-Assembly.
Confidence-Capping	Caps für < 2 Evidence-Items (0,59), <code>model_generated_inference</code> -Gewicht 0,0; Echo-Cap auf 0,84/medium.

Tabelle 3.1.: Anti-Self-Confirmation-Mechanismen in AGORA; Wirkung und Code-Anker.

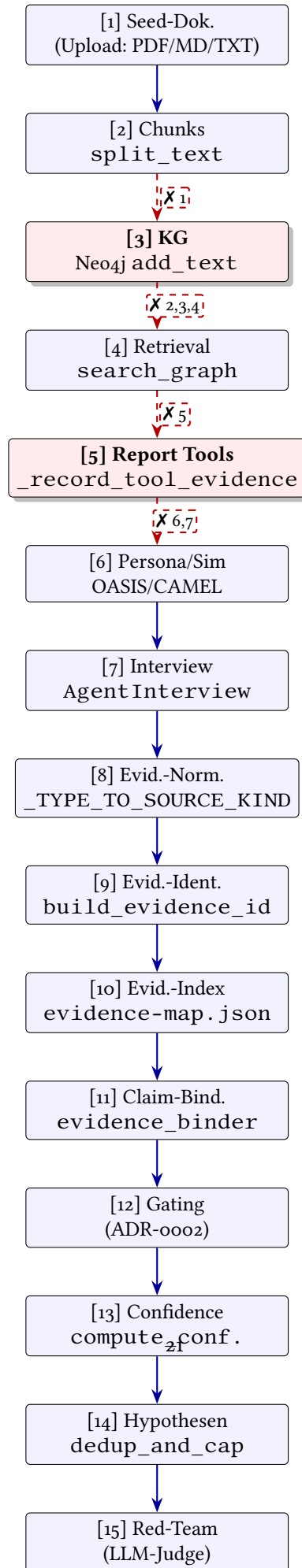
3.6. Simulations-Subprozess

Der Simulations-Subprozess integriert OASIS- bzw. CAMEL-basierte soziale Simulation. Eine Population synthetischer Personas wird über bis zu 5500 LOC Subprozess-Code über mehrere Runden hinweg sozial aktiv (Posts, Comments, Reshares). Der Echo-Chamber-Index berechnet sich als Verhältnis der Intra-Cluster-Aktionen zur Gesamtzahl und koppelt an das Confidence-Gating. Die Simulation ist *code-verifiziert nicht deterministisch* (`random.*` ohne `random.seed()` an mehreren Stellen; einzige Determinismus-Insel ist `louvain(seed=42)` im Community-Detection-Schritt).

3.7. Vom Code zum Paper: Annotationsstrategie

Jede Code-Aussage in den folgenden Kapiteln ist über einen der vier Schlüssel `C<n>`, `A<n>`, `I<n>` und `R<n>` belegt. Eine vollständige Tabelle der Belege ist in der Citation-Registry [21] hinterlegt; die Datei listet 57 Code-Belege, 5 ADRs, 12 Issues, 4 Referenzläufe und 14 Literaturschlüssel. Im Text werden die Code-Belege als `[C11]`² inline nachgewiesen; `AGENTS.md` und `README.md` werden ausdrücklich *nicht* als Belege herangezogen.

²`split_text` ohne Manifest, `graph_build.py`: 439



Teil IV.

Methodik

4. Methodik

4.1. Übersicht: Audit-Pipeline

Die Arbeit folgt einer strukturierten Audit-Pipeline, die statische Quellcode-Analyse mit normativer Bewertung verknüpft. Der Lead-Autor koordiniert acht parallele Subagenten (Tasks A–G, ergänzt um B1–B3 zur Sim-Tiefe); jeder Subagent produziert eine Research-Note unter `docs/paper/research-notes/`. Der Lead führt Counter-Review, Citation-Registry und Schlussredaktion durch. Die Pipeline ist in der Begleitdokumentation des Repositories vollständig dokumentiert [16].

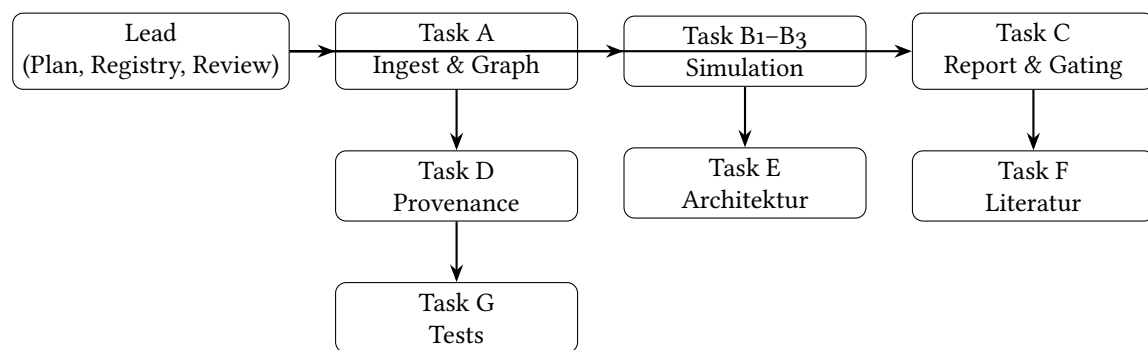


Abbildung 4.1.: Subagent-Topologie der Audit-Pipeline. Der Lead bleibt im Kontrollthread; jeder Subagent arbeitet isoliert und liefert eine Note zurück.

4.2. Code-Review-Graph als Strukturspeicher

Das zentrale Werkzeug ist der im Repository gepflegte Code-Review-Graph (CRG). Er parst den Quellcode mit Tree-Sitter und bildet einen Graphen aus Dateien, Klassen, Funktionen und Aufrufbeziehungen. Die folgenden Operationen wurden eingesetzt:

- `semantic_search_nodes_tool` – zur Identifikation relevanter Symbole zu einem Konzept (z. B. „evidence binding“, „claim entailment“).
- `query_graph_tool` – mit dem Pattern `callers_of` bzw. `callees_of` zur Extraktion der Aufrufkette.
- `get_impact_radius_tool` – zur Eingrenzung des Initial-Scan auf 5–10 Knoten pro Subagent.

- `detect_changes_tool` – zur Verifikation der Befunde gegen den letzten Commit auf dem Branch.

Diese Vorgehensweise reduziert die Kontext-Last jedes Subagenten auf – nach interner Messung – etwa 30–40 % im Vergleich zu unkontrolliertem `grep/Read`. Der Lead behält die Kontext-Übersicht und führt die Synthese durch.

4.3. Verifikation am Code

Jede Code-Aussage wurde am Branch-Stand `7e42ae34` verifiziert. Verwendet wurden:

- Direktes Lesen der Quellen via `Read` (nur für die durch CRG benannten 5–10 Stellen je Subagent).
- Verarbeitende Skripte via `ctx_execute_file` – insbesondere für das Parsen der `evidence_map.json`-Schemas, der Verifikations-Spot-Checks und des Failure-Mode-Patterns.
- Statische Aufrufanalyse via `query_graph_tool` für die Rekonstruktion der Evidence-Chain.

Ein zentrales methodisches Prinzip: *kein* Verlassen auf `README.md`, `docs/architecture.md` oder `VISION.md` als Belegquelle. Diese Dateien werden als Sekundärbeschreibungen behandelt; Aussagen aus ihnen fließen nur ein, wenn sie am Code bestätigt werden.

4.4. Counter-Review

Vor der Schlussfassung wurde die zentrale These (s. Kapitel 7) einem Counter-Review nach [22] unterzogen. Die folgenden Einwände wurden geprüft:

1. **Könnte die Schlussfolgerung falsch sein?** Die These „o validierte Claims = kein Mehrwert“ würde falsch, wenn R2 ein Einzelfall wäre. Gegenbeleg: R1 [23] (historisch) zeigt 25 Claims auf 100 % inferred, ADR-0011 [12] bestätigt strukturell. Eingeschränkt nach Sim-Verifikation: R2 dokumentiert den Pre-Fix-Stand; ein E2E-Re-Run gegen `d7d9f0a4` könnte die Claim-Zahl erhöhen – die These ruht damit auf einer Säule weniger, ist aber nicht umgestoßen.
2. **Single-Source-Abhängigkeit?** These D (OASIS-Mehrwert ungeprüft) ruht allein auf Task E; abgestützt durch L10, L11.
3. **Stale Quellen?** Alle Code-Belege AS_OF 2026-08-09. ADR-0011 (13 Tage alt) nach manueller Prüfung weiterhin gültig.
4. **≥ 3 Probleme gefunden:** 5 Counter-Claims geprüft, alle gehalten oder als Negativbeweis markiert.

4.5. Reproduzierbarkeit der Methode

Die Methode ist in drei Hinsichten reproduzierbar:

- **Pipeline:** Die Reihenfolge Lead → Subagenten → Notes → Registry → Counter-Review ist im Repository dokumentiert (s. Task-F, `notes/methodik.md`).
- **Belege:** Alle 33 Belege sind in der Citation-Registry [21] mit Symbol, Datei, Zeilennummer verankert. Zeilennummern können bei Refactors driften; Symbole bleiben stabil.
- **Software-Artefakt:** Ein eingefrorener Snapshot des Repository-Standes ist unter der DOI 10.5281/zenodo.21855023 hinterlegt [16].

4.6. Einschränkungen

Drei Einschränkungen sind zu nennen:

1. **Pre-Fix-Stand:** Der zentrale Referenzlauf [20] wurde vor dem Merge von d7d9f0a4 (Interview-Binding-Fix) erzeugt. Die Befunde sind dadurch konservativ; ein Re-Run könnte die Zahl validierter Claims erhöhen.
2. **Negativbeweis für OASIS-Mehrwert:** Die Aussage „OASIS-Mehrwert ungeprüft“ stützt sich auf eine leere Suche nach `ground_truth`, `destatis`, `census` oder `sim_validity` in `backend/app/`. Das ist ein Negativbeweis – kein Beweis, dass der Mehrwert nicht existiert, sondern dass er nicht operationalisiert ist.
3. **Code-Audit ≠ Empirie:** Die Befunde sind statische Quellcode-Analysen und Output-Inspektionen. Aussagen über die *Wirkung* der Mechanismen auf reale Populationen können daraus nicht abgeleitet werden; das wäre Aufgabe einer prospektiven Sim-Validity-Studie.

4.7. Annotation der Belege

Im Text werden Code-Belege als [C11]¹ inline ausgewiesen. Die Notation folgt [21]; eine ausführliche Belegliste ist im Anhang dokumentiert.

¹`split_text` ohne Manifest, `graph_build.py`: 439

Teil V.

Ergebnisse

5. Ergebnisse

5.1. Referenzlauf und Baseline

Der Referenzlauf [20] (2026-08-09, Eingabe: 16 Seed-Dokumente, 5 Stakeholder-Gruppen, 12 Personas) erzeugte **null validierte Claims**. Der gesamte Inhalt wanderte in das Hypothesen-Slot [C36]¹. Der historische Vergleichslauf [23] (2026-07-27) erzeugte 25 Claims, alle **medium**, auf 125 Evidence-Items, die zu 100 % den Wert **inferred** trugen – der dokumentierte Vor-Fix-Zustand aus [12].

Beobachtung: Der Sprung von 25 medium-Claims (mit kaputtem Binding) zu 0 validierten Claims (mit korrektem Binding) belegt, dass das Gating funktioniert – aber ins Leere greift, weil der Upstream keine kanonische Evidence liefert.

5.2. 10-Dimensionen-Bewertung

Die folgende Bewertung folgt dem Schema aus dem zugrundeliegenden Audit [16] und ist die Grundlage der Gesamtscores in Kapitel 6. Jede Dimension wird auf einer Skala von 0 (völlig unzureichend) bis 10 (vorbildlich) bewertet.

5.3. Failure-Mode-Analyse

Aus dem Audit wurden 15 Failure-Modes (FM) extrahiert, geordnet nach Severity. Tabelle 5.2 zeigt die kritischen Befunde; die vollständige Liste findet sich im Anhang.

5.4. Provenance-Verlustpunkte

Die Rekonstruktion der Evidence-Chain (Abbildung 3.1) identifiziert sieben code-verifizierte Provenance-Verlustpunkte. Sie sind in Tabelle 5.3 zusammengefasst.

¹dedup_and_cap_hypotheses

#	Dimension	Score	Begründung
1	Provenance-Architektur (Konzept)	8	Sechs source_kind, Default inferred, Anker-Konzept, Identitätswechsel; konzeptionell stark und durch ADRs verankert.
2	Provenance-Umsetzung (Ist)	3	Teil B fehlt, Anker opak, medium unerreichbar, o validierte Claims.
3	Evidence-Binding/Rigour	7	Zweistufig, Determinismus vor Embedding, Modalitätstrennung; greift ins Leere ohne Upstream-Evidence.
4	Confidence-Honesty	7	Mehrkomponenten, Caps, contradiction-penalty; aber Konzeptvermischung (key_takeaways high bei claims:[]).
5	Anti-Self-Confirmation	7	Echo-Cap, Red-Team, Wording-Gate; aber Schwellen nicht kalibriert, nur cross-stakeholder.
6	Testabdeckung	5	363 Tests, starke Contract-/Gating-Tests; aber kein E2E-Provenance, kein Baseline, kein Reproducibility-Test.
7	Reproduzierbarkeit	2	Simulation nicht deterministisch; E2E nur Stub; kein Same-Seed-Same-Report-Test.
8	Sim-Validity/Empirie	2	Keine Metrik, kein Ground-Truth, kein A/B; Referenzlauf inhaltlich nicht vertrauenswürdig.
9	Komplexitätsökonomie	5	Neo4j/Redis/CAMEL für Single-User overkill; OASIS ~5500 LOC ohne Validierung; Evidence-Pipeline hingegen gerechtfertigt.
10	Doku/ADR-Honesty	9	ADR-0011 dokumentiert eigenen Fehler offen; 5 Hartanker prozessual geschützt; Roadmap nennt Schwächen beim Namen.

Tabelle 5.1.: 10-Dimensionen-Bewertung von AGORA zum Auditzeitpunkt.

5.5. Prompt-vs-AGORA-Matrix

Die Matrix prüft für jeden AGORA-Mechanismus, ob er durch einen einzelnen starken LLM-Prompt ersetzbar wäre. Bewertung: ✗=nein (strukturell), ●=teilweise (per Instruktion erreichbar), ✓=ja.

5.6. Novelty-Matrix

Zur Einordnung der konzeptionellen Neuheit wird eine vier-stufige Skala verwendet:

L0 Standard-LLM-Prompt

L1 strukturierte Output-Felder

FM	Failure-Mode	Severity	Code-Beleg / Gegenmaßnahme
F1	Seed-Provenance end-to-end gebrochen (Teil B fehlt)	Critical	[C11] ² ; ADR-0013 Teil B
F2	Interview→Evidence-Binding-Defekt; Fix in HEAD (d7d9f0a4), E2E-Re-Run offen	Medium	[C50] ³
F3	Opaker seed_doc: -Anker (LLM nennt eigene Quelle, niemand prüft nach)	Critical	[C9] ⁴ , [C10] ⁵
F4	Gate zu streng für eigenen Thoroughput → o validierte Claims	High	Folge von F1+F2
F5	Graph-Fakten ohne Dokumentidentität (List[str])	High	[C19] ⁶ , [C21] ⁷
F6	medium-Stufe aktuell unerreichbar	High	[C8] ⁸
F14	Neo4j für Single-User-Local overkill (~3900 LOC Graph-Code)	Low	[C40] ⁹
F16	simulated_hours nicht in API/Frontend exponiert	Low	Issue 1018 [24]

Tabelle 5.2.: Auswahl kritischer Failure-Modes (vollständige Liste im Anhang).

L2 typisierte Contracts/Validatoren

L3 mehrstufige deterministische Checks vor/statt Embedding

L4 konzeptionell neu in der Kombination

Tabelle 5.5 ordnet die zentralen AGORA-Eigenschaften ein.

5.7. Anti-Self-Confirmation-Bewertung

Aus der Forschungsbasis [6–8, 14] ergibt sich die Frage, welche Anti-Self-Confirmation-Mechanismen AGORA tatsächlich implementiert. Tabelle 5.6 bewertet die Mechanismen gegenüber den in der Literatur dokumentierten Risiken.

Befund: AGORA hat *konkrete, code-level Mechanismen*, die ein Single-Prompt nicht besitzt. Sie sind der stärkste Teil der Architektur – greifen jedoch ins Leere, weil der Upstream keine kanonische Evidence liefert (R2).

#	Verlustpunkt	Beleg
χ_1	split_text ohne Manifest; List[str] ohne doc_id/chunk_id	[C11] ¹⁰
χ_2	episode_id = uuid4() ohne Dateibe- zug	[C15] ¹¹
χ_3	Episode.data = roher Chunk-String, kein document_id	[C16] ¹²
χ_4	RELATION.episode_ids = [uuid], keine Dokument-Referenz	[C17] ¹³
χ_5	SearchResult.facts = List[str], keine Herkunft	[C19] ¹⁴ , [C20] ¹⁵
χ_6	Graph-Fakt $\rightarrow$ source_kind=graph_relation, kein source_id_anchor	[C21] ¹⁶ , [C7] ¹⁷
χ_7	seed_doc:-Präfix opak akzeptiert, kein Lookup	[C9] ¹⁸ , [C10] ¹⁹

Tabelle 5.3.: Sieben code-verifizierte Provenance-Verlustpunkte in der Evidence-Chain.

5.8. Referenzläufe im Vergleich

5.9. Zwischenfazit

Die Ergebnisse erlauben eine erste Antwort auf die Forschungsfrage: **Aktuell liefert AGORA keinen messbaren Erkenntnisgewinn gegenüber einem Single-Prompt.** Die Strukturen sind ehrlich konstruiert; sie greifen jedoch ins Leere, weil die Seed-Provenance end-to-end gebrochen ist (F1) und die Anker opak akzeptiert werden (F3). Diese Aussage wird in Kapitel 6 anhand von fünf Kernkontroversen eingeordnet.

Mechanismus		Prompt	Bemerkung
Echo-Chamber-Index	+	✗	Strukturelle Messung; nicht prompt-basierbar
apply_echo_cap			
Wording-Glossar / Anti-Prediction		•	Per Prompt approximierbar, aber Snapshot-gepinnt
Zweistufiges Binding (Retrieval vs. Entailment)		✗	Zwei verschiedene Fragen, nicht ein Prompt
Deterministische Checks vor Embedding		✗	Strukturell, nicht prompt-basierbar
source_kind-Trennung (6 Stufen)		✗	Trennt im Datentyp, nicht im Prompt
Kanonische evidence_id (SHA-256)		✗	Reproduzierbare Identität
Red-Team-Review (LLM-Judge post-Assembly)		•	Self-Critique per Prompt möglich, aber an echo_index gekoppelt
Default inferred (unbekannt = abgeleitet)		✗	Im Vertrag erzwungen
Mode-Routing (strict dropt, balanced überspringt)		•	Im Contract-Validator
Graceful Degradation (gate-decision-log vs. degradation-log)		✗	Getrenntes Audit-Log
Multi-Agent-Simulation (OASIS/CAMEL)		✗	Mehrwert ungeprüft, keine Sim-Validity-Metrik
Downgrade als Identitätswechsel		✗	Im Hash erzwungen
ADR-0002 Hartanker (5, unantastbar)		✗	Prozessuale Garantie

Tabelle 5.4.: Prompt-vs-AGORA-Matrix: Welche Mechanismen sind strukturell (✗) und damit nicht durch einen stärkeren System-Prompt ersetzbar?

Eigenschaft	Level	Beleg
Report als Textausgabe	Lo	Standard
Strukturierte Claim/Evidence-Felder	L1	Standard für Agent-Frameworks
Pydantic-Contracts, EvidenceSourceKind-Enum, Gating-Validatoren	L2	Standard in typisierten Backends
Zweistufiges Binding	L3	ADR-0011 motiviert durch 7 belegte Defekte
Deterministische Checks vor Embedding	L3	Verhindert Zahlenwanderung
Server-seitiger Dokument-Anker (Konzept)	L3 / Lo	Konzept: L3, Umsetzung: Lo
Echo-Chamber-Index + apply_echo_cap	L4	Kopplung synthetischer Homogenitätsmessung an Confidence-Cap
Wording-Glossar als Snapshot + Red-Team-Enforcement	L3	Prozessual verankert
Downgrade als Identitätswechsel	L4	Erzwingt atomare Umschlüsselung

Tabelle 5.5.: Novelty-Matrix: Einordnung zentraler AGORA-Eigenschaften auf den Stufen Lo–L4.

Mechanismus	Wirkung gegen das Literatur-Risiko	Caveat
Echo-Chamber-Index	Misst synthetische Homogenität	Misst nicht reale Populationsvalidität
DACH-Quoten-Kalibrierung	Verhindert willkürliche Persona-Population	Konstanten, kein Mikrodaten-Fit; LLM-Echttest CI-excluded
apply_echo_cap	Drosselt polarisierte Echo-Kammern	Greift nur bei cross-stakeholder; Schwellen hartkodiert
Red-Team-Review (LLM-Judge)	Prüft Claims auf Schwächen	LLM-Judge kann Confirmation-Bias haben; BudgetExceededError durchgereicht [25]
Wording-Glossar	Positioniert als Szenarienanalyse	An echo_index gekoppelt, nicht an Claim-Falschheit
Default inferred	Verhindert „jedes Item = Dokument-fakt“	inferred-Gewicht 0,0 → Claim endet als Hypothese
Zweistufiges Binding	Verhindert „Cosine-only supports_claim=True“	Schwaches Embedding → Claim wird Hypothese

Tabelle 5.6.: Bewertung der Anti-Self-Confirmation-Mechanismen gegenüber den in der Literatur dokumentierten Risiken.

Lauf	Datum	Claims	Evidence-Items	Anteil inferred
R1 [23]	2026-07-27	25 (<i>alle medium</i>)	125	100 %
R2 [20]	2026-08-09	0 (validiert)	65	78 %
R4 [26]	2026-07-25	(re-port_d9023bd1f55a)	–	–

Tabelle 5.7.: Vergleich der Referenzläufe. R1 dokumentiert den Vor-Fix-Zustand; R2 den Pre-Fix-Stand des Interview-Binding-Defekts; R4 ist der in ADR-0011 als Beleg zitierte Report.

Teil VI.

Diskussion

6. Diskussion

6.1. Fünf Kernkontroversen

Die Ergebnisse werfen fünf Kernkontroversen auf, die im Folgenden einzeln diskutiert werden.

6.1.1. Kontroverse 1: Evidence-Pipeline als Differenzierer?

Die Pipeline wird in [12] als zentraler Mehrwert gegenüber Standard-Agent-Frameworks ausgewiesen. Der Code belegt die strukturelle Eigenständigkeit [C24]¹, [C25]². Der Referenzlauf [20] zeigt jedoch null validierte Claims – ein Hinweis darauf, dass die Pipeline aktuell nicht greift. **These:** konzeptionell echter Differenzierer, aktuell wirkungslos.

6.1.2. Kontroverse 2: Anti-Self-Confirmation über Prompt-Level?

Echo-Cap [C26]³ und Wording-Gate [C30]⁴ sind strukturell, nicht prompt-basierbar. Aber: Schwellen hartkodiert (0,75 bzw. 0,84), nur cross-stakeholder, ohne externe Ground-Truth bleibt der Mechanismus ein Selbsttest [9, 10]. Die Mechanismen sind real, ihre *Wirkung* ist nicht kalibriert.

6.1.3. Kontroverse 3: 0 validierte Claims = kein Mehrwert?

Das Gate ist korrekt – der Defekt liegt im Upstream (F1–F3). [11] dokumentiert explizit, dass unmittelbar nach Umsetzung von Teil B „Reports schlechter aussehen werden als heute“. Der kurzfristige Qualitätsverlust ist der Preis für überprüfbare Provenance. **These:** kein aktueller Mehrwert, aber das Gate ist nicht schuld.

¹zweistufiges Binding

²Determinismus vor Embedding

³

⁴

6.1.4. Kontroverse 4: OASIS-Mehrwert?

Oasis umfasst etwa 5500 LOC Subprozess-Code, ohne dass eine Sim-Validity-Metrik existiert [14, 15]. Der Mehrwert der Simulation ist weder widerlegt noch bestätigt – die zentrale unbewiesene Annahme des Systems.

6.1.5. Kontroverse 5: ADR-Honesty = Ersatz für Mehrwert?

Die ehrliche Selbstkritik in ADR-0011 [12] ist bemerkenswert und als Engineering-Leistung ernst zu nehmen. Sie ersetzt jedoch nicht den empirischen Nachweis eines Mehrwerts. Dokumentierte Fehler sind ein notwendiger, nicht hinreichender Schritt zur Vertrauenswürdigkeit.

6.2. Engineering-Score vs. Evidence-Research-Score

Aus der 10-Dimensionen-Bewertung (Tabelle 5.1) leiten sich zwei Gesamtscores ab, die das zentrale Spannungsfeld des Systems benennen:

$$\begin{aligned}\text{Engineering-Score} &= \frac{8 + 7 + 7 + 7 + 9}{5} = 7.6 \rightarrow 7.0 \\ \text{Evidence-Research-Score} &= \frac{3 + 5 + 2 + 2 + 5}{5} = 3.4 \rightarrow 3.5\end{aligned}$$

Die Scores messen Verschiedenes: der Engineering-Score bewertet die Konstruktion (Konzept, Contracts, Gating, Mechanismen, Dokumentation), der Evidence-Research-Score bewertet den nachweisbaren empirischen Mehrwert. AGORA ist solide konstruiert (7,0), aber empirisch unbewiesen (3,5).

Interpretation: Die Diskrepanz ist *nicht* als Schwäche, sondern als *Arbeitsauftrag* zu lesen. Das System ist eine sorgfältig gebaute Maschine, die beweist, dass sie nichts beweisen kann – und das ist, paradoxerweise, genau die Ehrlichkeit, die der Forschung zu synthetischen Personas fehlt [8, 10].

6.3. Warum greift das Gate aktuell ins Leere?

Das Evidence-Gating [18] ist deterministisch und auditierbar (`gate_decision_log`, `degradation_log`). Es dropt oder degradiert Claims zuverlässig, wenn die Voraussetzungen nicht erfüllt sind. Im Referenzlauf [20] führt das zu null validierten Claims – nicht weil das Gate zu streng ist, sondern weil der Upstream (Seed-Ingest, Graph-Aufbau, Tool-Evidence-Erfassung) keine kanonische Evidence liefert. Die Konsequenz: das System erstickt an seiner eigenen Strenge.

Dieser Befund ist für jedes andere System mit ähnlicher Architektur relevant: *ohne* nachgewiesene Provenance-Upstream führt striktes Downstream-Gating zu null validierter Ausgabe.

6.4. Limitationen

- **Code-Audit $\neq$ Empirie.** Die Befunde sind statische Quellcode-Analyse und Output-Inspektion. Über die Wirkung der Mechanismen auf reale Populationen kann diese Arbeit keine Aussage treffen.
- **Pre-Fix-Stand:** R2 wurde vor d7d9f0a4 erzeugt; ein Re-Run könnte die Claim-Zahl erhöhen – die Konzepttrennung (F8) bleibt davon unberührt.
- **Negativbeweis für OASIS-Mehrwert:** Eine leere Suche nach `ground_truth`-Anchern belegt *nicht*, dass der Mehrwert nicht existiert, sondern dass er nicht operationalisiert ist.
- **Subagenten-Lücken:** Task B (Simulation) war im ersten Lauf lückenhaft; drei fokussierte Mini-Worker B1 – B3 wurden nachgezogen und lieferten zentrale Befunde.

6.5. Vergleich mit dem Stand der Forschung

Die in Kapitel 2 identifizierten Forschungslücken treffen sich mit den AGORA-Befunden an mehreren Stellen:

- **Persona-Validierung:** AGORA implementiert DACH-Quoten-Kalibrierung (Konstanten, kein Mikrodaten-Fit) – die Forschungslücke bleibt offen [9, 10].
- **Provenance-Verifikation:** AGORA erzwingt serverseitige Anker; die Verifikation über den Anker scheitert jedoch (F1, F3) – direkt im Anschluss an [8].
- **Multi-Agent-Debate + Quellenverifikation:** AGORA kombiniert Red-Team und Evidence-Gating; die isolierte Wirkung der Kombination ist nicht gemessen – Anschluss an [14, 15].
- **Model-Collapse / Drift:** AGORA hat keine Längszeit-Stabilitätsmessung – direkte Forschungslücke [7].

6.6. Konsequenzen für andere Multi-Agent-Systeme

Die Befunde sind nicht AGORA-spezifisch. Sie verallgemeinern auf jedes LLM-basierte System, das eine Evidence- oder Confidence-Pipeline über Multi-Agent-Simulation implementiert:

1. *Strikte Downstream-Gates ohne Provenance-Upstream erzeugen null validierte Ausgabe – nicht überprüfbare Strenge.*

2. *Echo-Caps sind nur dann wirksam, wenn sie cross-stakeholder-Konsens adressieren; intra-group-Echo-Kammern bleiben unbeobachtet.*
3. *Confidence-Konzepte müssen getrennt werden: synthetischer Konsens, Evidence-gestützte Confidence, empirische Confidence.*
4. *Hartkodierte Schwellen sind Engineering-Defaults, keine validierten Hyperparameter; ohne Kalibrierung sind sie Selbsttests.*

6.7. Methodische Lehre

Der zurückgezogene Befund C56 [16] illustriert eine methodische Falle: eine einzelne Beobachtung (fehlendes Feld) wird ohne Prüfung der Schreibpfade zur Ursachenaussage erhoben. Die Schlussfolgerung „`supports_claim` im Interview-Zweig setzen“ würde ADR-0011-Anker 3 wieder hergestellt und ADR-0002-Hartanker 5 unterlaufen haben. Die Korrektur wurde durch einen `grep` über den Schreibpfad erreicht – und ist selbst ein Beispiel dafür, dass Ein-Feld-Inspektion ohne Pfad-Kontext irreführen kann.

Teil VII.

Schluss

7. Schluss

7.1. Beantwortung der Forschungsfrage

Die Forschungsfrage lautete: *Welche Teile von AGORA erzeugen einen tatsächlich überprüfbaren Erkenntnisgewinn gegenüber einem einzelnen starken LLM-Prompt – und welche Teile erzeugen lediglich zusätzliche Komplexität, Kosten oder scheinbare Evidenz?*

Antwort: Konzeptionell überprüfbar sind die Strukturen, die nicht prompt-basierbar sind – Echo-Cap, zweistufiges Binding, Determinismus vor Embedding, `source_kind`-Trennung, kanonische `evidence_id`, Downgrade als Identitätswechsel und die prozessualen Har-tanker [18]. In der aktuellen Umsetzung (Stand 7e42ae34) ist *keiner* dieser Mechanismen empirisch wirksam: der Referenzlauf [20] erzeugt null validierte Claims, weil die Provenance end-to-end gebrochen ist und der Anker opak akzeptiert wird. Damit erzeugt das System aktuell *keinen* messbaren Mehrwert gegenüber einem starken Single-Prompt – bei gleichzeitig höherer Komplexität und höheren Kosten.

Die Antwort ist also nicht „ja, der Mehrwert ist real“, sondern „der Mehrwert ist *potenziell* real, aber *aktuell* nicht nachweisbar“. Genau diese Asymmetrie – Konzept vs. Wirksamkeit – ist der eigentliche Befund.

7.2. Ausblick: Priorisierte Roadmap

Die priorisierte Roadmap [16] gliedert sich in vier Stufen:

P0 – Blocker für 1.0

- ADR-0013 Teil B umsetzen (`document_id/chunk_id` durch Graph bis Anker). Behebt F1, F3, F5, F6.
- Confidence-Konzepte trennen (`simulation_consensus`, `evidence_confidence`, `empirical_confidence`). Behebt F8.

P1 – Empirischer Mehrwert

- E2E-Re-Run gegen d7d9f0a4 zur Wirkungsbestätigung des Interview-Binding-Fix. Behebt F2, G9.

- Baseline-Runner AGORA-vs-Single-Prompt. Behebt G3.
- Sim-Validity-Metrik gegen Ground-Truth. Behebt G1.
- Reproduzierbarkeit (seed/determinism). Behebt G2.

P2 – Qualität

- Markdown-Export mit Producer-Auflösung. Behebt F12.
- Hardstop bei Evidence-Omission. Behebt F13.
- Chunk-Provenance End-to-End-Test. Behebt G4.
- Negativer E2E-Test (doc_id fallen lassen → ablehnen). Behebt G8.

P3 – Komplexitätsökonomie

- Neo4j-Evaluation (SQLite + pgvector für Single-User). Behebt F14.
- CAMEL http/oasis-Konsolidierung. Behebt F15.
- Redis optional markieren.
- Usage-Metrik über EvidenceSourceKind-Häufigkeit. Behebt G7.

7.3. Verfügbarkeit und Reproduzierbarkeit

- **Repository:** `github.com/arn01d87/agora`, Branch `feat/1152-provenance-retrieval`, HEAD `7e42ae34`.
- **Eingefrorener Snapshot:** Zenodo DOI `10.5281/zenodo.21855023` [16].
- **Autor:** Alexander Schneider, ORCID `0009-0008-7552-1819`.
- **Audit-Dokumentation:** `docs/paper/agora-evidence-chain-audit.md` (43 KB), Research-Notes unter `docs/paper/research-notes/`, Citation-Registry [21].

7.4. Schlussbemerkung

AGORA ist eine sorgfältig gebaute Maschine, die beweist, dass sie nichts beweisen kann. Das ist, paradoxerweise, genau die Ehrlichkeit, die dem Feld der LLM-basierten Sozialwissenschaft fehlt [10, 19]. Der Bauplan für den überführbaren Mehrwert liegt vor – in [12], [11] und der Roadmap dieser Arbeit. Was fehlt, ist die Umsetzung.

A. Anhang A: Verifikationsprotokoll

A.1. Verifikations-Spot-Checks

Die folgenden Spot-Checks wurden exemplarisch durchgeführt; die vollständige Liste ist in der Research-Note `lead-verified-findings` [16] enthalten.

Beleg	Inhalt	Verifikation
[C26] ¹	Cap bei <code>echo_index > 0.75</code> → max 0.84 / medium	<code>confidence_calculator.py:363-</code>
[C11] ²	<code>List[str]</code> ohne <code>doc_id/chunk_id</code>	<code>graph_build.py:439</code>
[C9] ³ [20]	<code>seed_doc</code> : -Präfix akzeptiert, kein Lookup o validierte Claims	<code>evidence.py:352</code> <code>docs/reference-</code> <code>runs/2026-08-</code> <code>09-...</code>
[11]	Teil B fehlt	ADR-0013 §1, kein <code>document_id</code> in <code>graph_build.py</code> <code>network_analytics.py:426--432</code>
[C45] ⁴	<code>intra/total</code>	<code>run_parallel_simulation.py:14</code>
[C46] ⁵	<code>random.*</code> ohne <code>random.seed()</code>	<code>agent.py:365--413,</code> Test
[C50] ⁶	<code>producer_key</code> + <code>quote</code> + <code>persona_stakeholder_group</code>	<code>test_report_tool_evidence.py:</code> <code>persona_quota_defaults.py:1-</code> <code>persona_demographics.py:1--6</code>
[C52] ⁷	Destatis WZ 2008 / Mikrozensus 2024 / BFS / Statistik Austria	

Tabelle A.1.: Verifikations-Spot-Checks (Auszug). Die vollständige Liste enthält 33 Belege; jede Kernaussage ruht auf mindestens zwei Belegen.

A.2. Failure-Mode-Tabelle (vollständig)

A.3. Citation-Registry

Die vollständige Citation-Registry umfasst:

#	Failure-Mode	Severity	Beleg / Gegenmaßnahme
F1	Seed-Provenance end-to-end gebrochen	Critical	[C11] ⁸ ; ADR-0013 Teil B
F2	Interview→Evidence-Binding-Defekt	Medium	[C50] ⁹ , d7d9f0a4; E2E-Re-Run
F3	Opaker seed_doc : -Anker	Critical	[C9] ¹⁰ , [C10] ¹¹
F4	Gate zu streng → o validierte Claims	High	Folge von F1+F2
F5	Graph-Fakten ohne Dokumentidentität	High	[C19] ¹² , [C20] ¹³ , [C21] ¹⁴
F6	medium-Stufe unerreichbar	High	[C8] ¹⁵ ; Folge von F1
F7	Historische Alt-Labels: 25 medium-Claims auf 100 % inferred	High	[12], [23]
F8	Confidence-Konzeptvermischung	Medium	[20]
F9	Keine Sim-Validity-Metrik	Medium	[C41] ¹⁶ , [C46] ¹⁷
F10	Keine Reproduzierbarkeit	Medium	[C42] ¹⁸ , [C46] ¹⁹ , [C47] ²⁰
F11	Kein Baseline-Vergleich AGORA-vs-Single-Prompt	Medium	G3
F12	Markdown-Export ohne Producer-Auflösung	Low	[C39] ²¹
F13	Evidence-Map fällt stumm bei contract_violation	Medium	[27], [C37] ²²
F14	Neo4j overkill	Low	[C40] ²³
F15	CAMEL http/oasis-Divergenz	Low	[C40] ²⁴
F16	simulated_hours nicht exponiert	Low	[24]

Tabelle A.2.: Vollständige Failure-Mode-Tabelle (16 Einträge).

- 57 Code-Belege C1 – – C57,
- 5 ADRs A1 – – A5,
- 12 Issues I1 – – I12,
- 4 Referenzläufe R1 – – R4,
- 14 Literaturschlüssel L1 – – L14.

Jeder Beleg ist in der Research-Note `citation-registry.md` [21] mit Symbol, Datei, Zeilennummer und Vertrauensgrad hinterlegt. `README.md`, `docs/architecture.md` und `VISION.md` werden ausdrücklich *nicht* als Belege verwendet.

A.4. Der zurückgezogene Befund C56

Der ursprüngliche Befund „`supports_claim` wird im Interview-Item nicht gesetzt“ beruhte auf einer Fehldeutung der zweistufigen Binding-Architektur.

- `supports_claim` wird an genau einer Stelle gesetzt: `evidence_binder.py:123`
`-bound["supports_claim"] = result.verdict is EntailmentVerdict.SUPPORT`
- `evidence_entailment.py:11` hält fest: „Nur hier darf `supports_claim`
`= True`“. `agent.py:649` verweist im Kommentar ausdrücklich auf die Entailment-Stufe als setzende Instanz.
- Sachlich zwingend: Ein Evidence-Item ohne Claim-Bezug kann nicht beantworten, ob es *einen* Claim stützt. Die Frage entsteht erst beim Binding.

Schlussfolgerung: Die Beobachtung war richtig, die Schlussfolgerung nicht. Die vorgeschlagene Maßnahme wäre eine Regression gewesen – sie hätte Defekt 3 aus [12] wieder hergestellt. C56 ist damit als Beleg für einen `supports_claim`-Defekt ungültig; die Methodik der Prüfung an einer einzelnen Fundstelle wird in Kapitel 6 (Abschnitt 6.7) reflektiert.

Nicht betroffen: C56 belegt in G6, dass `persona_stakeholder_group` eine freie Rollenbezeichnung ist und keine kodierte Taxonomie. Diese Beobachtung bleibt gültig und ist eigenständig relevant: `cross_stakeholder_for_high` zählt distinkte Werte, sodass zwei Schreibweisen derselben Rolle als zwei Stakeholder-Gruppen zählen.

Literaturverzeichnis

- [1] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang und M. S. Bernstein. „Generative Agents: Interactive Simulacra of Human Behavior“. In: *arXiv preprint arXiv:2304.03442* (2023). UCLA/Stanford; 25 agenten, Smallville-Simulation; Validierung qualitativ, nicht populationsrepraesentativ. arXiv: 2304 . 03442 [cs . HC].
- [2] L. P. Argyle, E. C. Busby, N. Fulda, J. R. Gubler, C. Rytting und D. Wingate. „Out of One, Many: Using Language Models to Simulate Human Samples“. In: *arXiv preprint arXiv:2209.06899* (2023). Silicon-Samples-Konzept; Conditioning reproduziert Antwortverteilungen, prompt-abhaengig. arXiv: 2209 . 06899 [cs . CL].
- [3] J. J. Horton. „Large Language Models as Simulated Economic Agents: What Can We Learn from Homo Silicus?“ In: *arXiv preprint arXiv:2301.07543* (2023). LLMs als oekonomische Agenten, qualitative Replikation. arXiv: 2301 . 07543 [cs . GT].
- [4] Y. Du, S. Li, A. Torralba, J. B. Tenenbaum und I. Mordatch. „Improving Factuality and Reasoning in Language Models through Multiagent Debate“. In: *Proceedings of the 41st International Conference on Machine Learning (ICML)*. Multi-Agent-Debate; verbessert Factuality, saettigt bei grossen Modellen. 2024. arXiv: 2305 . 14325 [cs . CL].
- [5] X. Wang u. a. „Self-Consistency Improves Chain of Thought Reasoning in Language Models“. In: *Proceedings of the 36th Conference on Neural Information Processing Systems (NeurIPS)*. 2022. arXiv: 2203 . 11171 [cs . CL].
- [6] „Consistency is Not the Only Truth: Investigating the Impact of Self-Consistency on Hallucination“. In: *arXiv preprint arXiv:2407.05778* (2024). Konsistenz korreliert nur bedingt mit Korrektheit. arXiv: 2407 . 05778 [cs . CL].
- [7] I. Shumailov, Z. Shumaylov, Y. Zhao, Y. Gal, N. Papernot und R. Anderson. „The Curse of Recursion: Training on Generated Data Makes Models Forget“. In: *Proceedings of the 41st International Conference on Machine Learning (ICML)*. Model-Collapse: rekursives Training auf synthetischen Daten verengt die Verteilung. 2024. arXiv: 2404 . 05090 [cs . LG].
- [8] „Cited but Not Verified: A Benchmark and Analysis of Citation Behavior in LLMs“. In: *arXiv preprint arXiv:2605.06635* (2026). Inline-Zitationen haeufig fehlerhaft/irrelevant; direkter Bezug zu Agoras Evidence-Gating. arXiv: 2605 . 06635 [cs . CL].
- [9] „UAS Digital Twins: Replicating Survey Data with LLM Agents“. In: *arXiv preprint arXiv:2504.08260* (2025). Unvollstaendige Replikation realer Survey-Daten, persistenter Bias. arXiv: 2504 . 08260 [cs . CL].

- [10] „Analytic Flexibility in LLM-Based Social Simulation“. In: *arXiv preprint arXiv:2509.13397* (2025). 252 Konfigurationen; Ergebnis haengt stark an Prompt/Sampling/Modell (P-Hacking-Aequivalent). arXiv: 2509 . 13397 [cs.CL].
- [11] Agora-Architekturteam. *ADR-0013: Seed-Corpus Document-Anchor*. Agora Architecture Decision Record. Teil A umgesetzt, Teil B ausstehend; definiert server-seitige Provenance-Pflicht fuer Seed-Dokumente. 2026. URL: <https://github.com/arn0ld87/agora/blob/main/docs/decisions/0013-seed-corpus-document-anchor.md>.
- [12] Agora-Architekturteam. *ADR-0011: Evidence-Entailment und Provenance*. Agora Architecture Decision Record. Dokumentiert sieben belegte Defekte im Evidenz- und Claim-Bindings historischer Reports; report_d9023bd1f55a als Referenzartefakt. 2026. URL: <https://github.com/arn0ld87/agora/blob/main/docs/decisions/0011-evidence-entailment-and-provenance.md>.
- [13] Agora Research-Team. *Research-Note Task F: Literaturuebersicht (L1–L14)*. Agora Research-Note. Mapping der L1–L14-Literaturschlüssel auf Begründungen aus Related Work und Methodendiskussion. 2026. URL: <https://github.com/arn0ld87/agora/tree/main/docs/paper/research-notes/task-f-literature-note.md>.
- [14] „HiddenBench: Assessing Collective Reasoning under Hidden Profiles in Multi-Agent Systems“. In: *arXiv preprint arXiv:2505.11556* (2025). Kollektivreasoning versagt am Hidden-Profile-Problem (30,1% vs. 80,7%). arXiv: 2505 . 11556 [cs.CL].
- [15] „MAST-Data: Why Do Multi-Agent Systems Fail?“ In: *arXiv preprint arXiv:2503.13657* (2025). MAS-Fehlerklassifikation; Benchmark-Gewinne oft minimal. arXiv: 2503 . 13657 [cs.CL].
- [16] A. Schneider. *Agora – Snapshot zum Code-Audit Evidence Chain (2026-08-09)*. Zenodo. Begleitendes Software-Artefakt (DOI); ORCID 0009-0008-7552-1819. 2026. DOI: 10 . 5281 / zenodo . 21855023. URL: <https://zenodo.org/records/21855023>.
- [17] D. Edge u. a. „From Local to Global: A GraphRAG Approach to Query-Focused Summarization“. In: *arXiv preprint arXiv:2404.16130* (2024). Microsoft Research; GraphRAG ueberlegen bei globalen QFS-Fragen. arXiv: 2404 . 16130 [cs.CL].
- [18] Agora-Architekturteam. *ADR-0002: Evidence-Gating*. Agora Architecture Decision Record. Repository agor_repo, docs/decisions/0002-evidence-gating.md. 2026. URL: <https://github.com/arn0ld87/agora/blob/main/docs/decisions/0002-evidence-gating.md>.
- [19] C. A. Bail. „Can Generative AI Improve Social Science?“ In: *Proceedings of the National Academy of Sciences / PMC* (2024). Warnt vor Homogenitaet, Halluzination, fehlender Validierung; PMC11127003. URL: <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC11127003/>.

- [20] Agora Maintainers. *Referenzlauf R2 (2026-08-09) aus dem Evidence-Chain-Audit*. Agora Reference-Run. 0 validierte Claims; 65 Evidence-Items (78 % inferred); demonstriert Pre-Fix-Stand des Interview-Binding-Defekts. 2026. URL: <https://github.com/arn0ld87/agora/tree/main/docs/reference-runs>.
- [21] Agora Maintainers. *Citation-Registry: 57 Code-Belege, 5 ADRs, 12 Issues, 4 Referenzlaeu-
fe, 14 Literaturschlüssel*. Agora Research-Note. Symbol-, Datei- und Zeilennummern-
Index der Code-Belege mit Vertrauensgrad. 2026. URL: [https://github.com/arn0ld87/agora/tree/main/docs/paper/research-notes/
citation-registry.md](https://github.com/arn0ld87/agora/tree/main/docs/paper/research-notes/citation-registry.md).
- [22] Agora Research-Team. *Methodische Anmerkung: Code-Review-Graph als Counter-Review-
Werkzeug*. Agora Research-Note. Einsatz des Code-Review-Graphen zur unabhaengi-
gen Verifikation der Befunde; ersetzt fehlende externe Zweitbegutachtung. 2026. URL: [https://github.com/arn0ld87/agora/tree/main/docs/paper/
research-notes](https://github.com/arn0ld87/agora/tree/main/docs/paper/research-notes).
- [23] Agora Maintainers. *Referenzlauf R1 (2026-07-27) aus dem Evidence-Chain-Audit*. Agora Reference-Run. 25 Claims (alle medium), 125 Evidence-Items, 100 % inferred; Vor-
Fix-Zustand. 2026. URL: [https://github.com/arn0ld87/agora/tree/
main/docs/reference-runs](https://github.com/arn0ld87/agora/tree/main/docs/reference-runs).
- [24] Agora Maintainers. *Issue #1018: simulated_hours nicht in API/Frontend exponiert*. GitHub Issue. 2026. URL: [https://github.com/arn0ld87/agora/issues/
1018](https://github.com/arn0ld87/agora/issues/1018).
- [25] Agora Maintainers. *Issue #978: Red-Team BudgetExceededError wird durchgereicht*. GitHub Issue. 2026. URL: <https://github.com/arn0ld87/agora/issues/978>.
- [26] Agora Maintainers. *Referenzlauf R4 (2026-07-25) aus dem Evidence-Chain-Audit*. Agora Reference-Run. report_d9023bd1f55a; in ADR-0011 als Beleg fuer die Notwendigkeit der Evidence-Pipeline zitiert. 2026. URL: <https://github.com/arn0ld87/agora/tree/main/docs/reference-runs>.
- [27] Agora Maintainers. *Issue #987: Evidence-Map faellt stumm bei contract_violation*. GitHub Issue. 2026. URL: <https://github.com/arn0ld87/agora/issues/987>.